JC4002 Knowledge Representation

Day 7: Description Logic

Dr Ferdian Jovan ferdian.jovan@abdn.ac.uk
Dr Aiden Durrant aiden.durrant@abdn.ac.uk
Dr Jiangtian Nie jiangtian.nie@abdn.ac.uk

Day 7
Description Logic

1. **Structured Descriptions**
2. A Description Language
3. Meaning and Entailment
4. Computing Entailment and Normalisation
5. Computing Satisfaction

Structured Descriptions

- On the previous lecture, we looked at knowledge organisation sorted by *facts* and *rules / definitions*.

- However, those representations had more focus on the procedures and inferences (i.e. resolution process) rather than about the actual objects, categories, and definitions themselves.

- Reasoning about objects in everyday thinking goes well beyond the simple cascaded computations seen in the last lecture, and is based on considerations like:

Structured Descriptions

1. **objects** naturally fall into **categories** (e.g., my pet is a dog, my wife is a physician), but are very often thought of as being members of **multiple categories** (e.g., I am an author, an employee, and a father)
2. categories can be **more general or more specific** than others (e.g., Westie and Schnauzer are types of dogs, a rheumatologist is a type of physician, a father is a type of parent);
3. in addition to generalization being common for categories with simple names, it is also natural for those with **more complex descriptions** (e.g., a part-time employee is an employee)
4. objects have parts, sometimes in multiples (e.g., books have titles, tables have legs, automobiles have wheels);
5. the **relationships** among an object's parts is essential to its being considered a member of a category (e.g., a stack of bricks is not the same as a pile of the very same bricks).

Structured Descriptions

- In FOL, all categories and properties of objects are represented by atomic predicates.
- In some cases, these correspond to simple *nouns* in English such as 'Person' or 'City'.
  - These categories of objects are usually represented by predicates with one argument e.g. Person(X), City(X).
- In other cases, the predicates seem to be more like *noun phrases* such as 'MarriedPerson' or 'UKCity'.
  - Intuitively, these predicates have internal structures and connections to other predicates.
  - A married person must be a person.
    - $\text{MarriedPerson}(X) = \forall x\,(\exists y\ \text{Married}(X, Y) \rightarrow \text{Person}(X))$
  - These connections hold by *definition* (by what we define what the predicate means), not by virtue of the facts we believe about the world.

Structured Descriptions

- Traditional first-order logic does not provide any tools for dealing with compound predicates of this sort. In a sense, the only noun phrases in FOL are the nouns. But given the prominence and naturalness of such constructs in natural language, it is worthwhile to consider knowledge representation machinery that does provide such tools.

- Because a logic that would allow us to manipulate complex predicates would be working mainly with *descriptions*, we call a logic system based on these ideas a *description logic* (DL)

Day 7
Description Logic

1. Structured Descriptions
2. **A Description Language**
3. Meaning and Entailment
4. Computing Entailment and Normalisation
5. Computing Satisfaction

Concepts, Roles, and Constants

- In description logic, there are *sentences / formulas* that will be true or false (as in FOL).
- In addition, there are three sorts of expressions that act like nouns and noun phrases in English.
  - ***concepts (classes)*** are like category nouns (i.e. groups of objects that share similar characteristics). Example: Dog, Teenager, Person, City
  - ***roles (relations)*** are like relational nouns (i.e. predicates that have more than one argument representing a relation between them). Example: :Age, :Parent
  - ***constants (instances)*** are like proper nouns (i.e. the instances of concepts). Example: JohnSmith, Aberdeen.
- These correspond to unary predicates, binary predicates, and constants (respectively) in FOL.
- However, unlike in FOL, concepts need not be atomic and can have semantic relationships to each other.
  - roles will remain atomic (for now).

A Description Logic

- Those three expressions are considered *nonlogical* symbols (those that are application-dependent) in DL:
  - A set I of instance names (or constants), written in uncapitalised mixed case, e.g., maryAnn, johnSnow, Aberdeen.
  - A set C of class names (or concepts), written in capitalised mixed case, e.g., Person, EnglandCity.
  - A set R of relation names (or roles), written like concepts, but preceded by a ":", e.g., :Child, :Height, :Arm
- The logical symbols in DL:
  - Punctuation:"[", "]", "(", ")";
  - Positive integers: 1, 2, 3, …;
  - Operators: "ALL", "EXISTS", "FILLS", "AND";
  - Connectives: "$\sqsubseteq$", "$\doteq$", "$\rightarrow$"

A Description Logic

- The set of concepts of DL is the least set satisfying the following:
  - every atomic concept is a concept;
  - if r is a role, and d is a concept, then [ALL r d] is a concept;
  - if r is a role, and n is a positive integer, then [EXISTS n r] is a concept;
  - if r is a role, and c is a constant, then [FILLS r c] is a concept;
  - if $d_1 \ldots d_n$ are concepts, then [AND $d_1 \ldots d_n$] is a concept
- Finally, there are three types of sentences / formulas in DL
  - If d is a concept and c is a constant, then d(c) is a sentence;
  - If r is a role, and c_1, c_2 are constants, then $r(c_1, c_2)$ is a sentence;
  - if d_1 and d_2 are concepts, then $(d_1 \sqsubseteq d_2)$ is a sentence;
  - if d_1 and d_2 are concepts, then $(d_1 \doteq d_2)$ is a sentence;
  - if d is a concept and c is a constant, then $(c \rightarrow d)$ is a sentence;

DL Syntax as d.e.l.g.

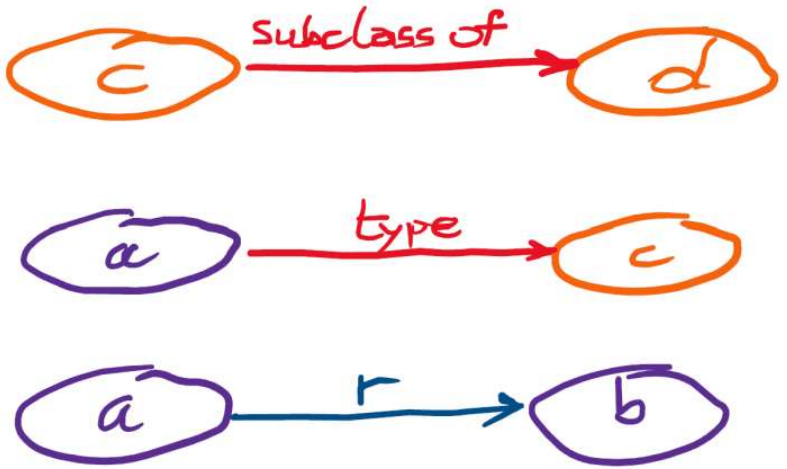
Sentences in DL syntax

$c \sqsubseteq d$

$c(a)$

$r(a,b)$

Directed **e**dge-**l**abelled **g**raph



DL Meaning

- Constants stand for individuals, concepts for sets of individuals, and roles for binary relations.
- If d_1 and d_2 are concepts, then ($d_1 \sqsubseteq d_2$) is a sentence;
  - ($d_1 \sqsubseteq d_2$) says that d_1 is subsumed by d_2.
  - This means that every instance of d_1 is also an instance of d_2.
- if d_1 and d_2 are concepts, then ($d_1 \doteq d_2$) is a sentence;
  - ($d_1 \doteq d_2$) says that d_1 is equivalent to d_2.
  - It is equivalent to ($d_1 \sqsubseteq d_2$) ^ ($d_2 \sqsubseteq d_1$)
- if d is a concept and c is a constant, then ($c \rightarrow d$) is a sentence;
  - ($c \rightarrow d$) says that an instance that satisfies c must be an instance of d.

Activity 1

In pairs, discuss the following question (5 minutes) and then share your answers with the rest of your colleagues (5 minutes):

"Is there any difference between $d_1 \sqsubseteq d_2$ and $d_1 \rightarrow d_2$"

DL Meaning

- [**EXISTS** n r] stands for the class of individuals in the domain that are related by relation r to at least *n* other individuals.
  - Example: [**EXISTS** 1 :Child] represents someone who is not childless
  - This translates to equivalent English "There must be at least one (person) that has a child". Note that we don't care about the name of the child (i.e. Child(X, _)), we are interested in the (instance) parent, hence child's name is anonymous.
- [**FILLS** r c] stands for constant c that is r-related.
  - Example: [**FILLS** :Cousin vinny] represents someone whose cousins is Vinny.
  - This translates to equivalent FOL ∃X Cousin(X, vinny), note that instance vinny fills the **second argument** of a relation Cousin.

DL Meaning

- [**ALL** r d] stands for those instances who are r-related only to elements of concept d.
  - In other words: for all instances x, if x is related to some instance y through the role r, then y must be an instance of the concept d.
  - FOL equivalent: ∀x ∀y (r(x, y) → d(y))
  - Example: [**ALL** :HasChild Person] describes every child (every individual related via 'hasChild') of a given individual is a Person.
  - The expression **ALL** restricts the range of the role r such that any individual related to the subject by this role must satisfy the concept d.
- [**AND** d_1 … d_n] stands for anything that is described by d_1 … d_n.
  - This expression defines a new concept that is the intersection of all the given concepts. An individual belongs to this new concept if and only if it belongs to each concept d_1 through d_n.
  - Note that d_1 … d_n are concepts, **not role**!

DL Examples

- With DL logical symbols, we can create complex concepts resulting into a complex sentence / formula.
- For example:

  [**AND** Wine

  [**FILLS** :Color red]

  [**EXISTS** 2 :GrapeType]]

- would represent the category of a blended red wine (literally, a wine, one of whose colors is red, and which has at least two types of grape in it).

Activity 2

In pairs, construct a DL sentence for the following:

“A company with at least 7 directors whose managers are all women with PhDs, and whose min salary is £35/hour”

A DL Knowledge Base

A DL KB is a set of DL sentences / formulas serving mainly to:

1. Gives names to definitions.

   (FatherOfDaughters $\doteq$ [**AND** Male [**EXISTS** 1 :Child] [**ALL** :Child Female]])

   "A FatherOfDaughters is precisely a male with at least 1 child and all of whose children are female"

A DL Knowledge Base

A DL KB is a set of DL sentences / formulas serving mainly to:

1. Gives names to definitions.

   (FatherOfDaughters $\doteq$ [**AND** Male [**EXISTS** 1 :Child] [**ALL** :Child Female]])

   "A FatherOfDaughters is precisely a male with at least 1 child and all of whose children are female"

2. Gives names to partial definitions.

   (Dog $\sqsubseteq$ [**AND** Mammal Pet CarnivorousAnimal [**FILLS** :VoiceCall barking]])

   "A dog is among other things a mammal that is a pet and a carnivorous animal whose voice call includes barking"

A DL Knowledge Base

A DL KB is a set of DL sentences / formulas serving mainly to:

1. Gives names to definitions.

   (FatherOfDaughters $\doteq$ [**AND** Male [**EXISTS** 1 :Child] [**ALL** :Child Female]])

   "A FatherOfDaughters is precisely a male with at least 1 child and all of whose children are female"

2. Gives names to partial definitions.

   (Dog $\sqsubseteq$ [**AND** Mammal Pet CarnivorousAnimal [**FILLS** :VoiceCall barking]])

   "A dog is among other things a mammal that is a pet and a carnivorous animal whose voice call includes barking"

3. Assert properties of instances / individuals.

   (joe $\rightarrow$ [**AND** FatherOfDaughters Surgeon]) "joe is a FatherOfDaughters and a Surgeon"

A DL Knowledge Base

- In a fully defined concept (e.g. FatherOfDaughters with $\doteq$), we have a set of necessary and sufficient conditions for being a FatherOfDaughters, exactly expressed by the right-hand side.
- If we used the $\sqsubseteq$ connective instead, the sentence would say only that FatherOfDaughters as a concept was subsumed by the ones on the right.
- Without a $\doteq$ sentence in the KB defining it, we consider FatherOfDaughters to be a primitive concept in that we only have necessary conditions it must satisfy.
- As a result, although we could draw conclusions about an individual FatherOfDaughters once we were told it was one (like joe), we would not have a way to recognize an individual definitively as a FatherOfDaughters.
    - For example: a mammal that is a pet and a carnivorous animal whose voice call includes barking does not necessarily mean it is a dog.

Day 7
Description Logic

1. Structured Descriptions
2. A Description Language
3. **Meaning and Entailment**
4. Computing Entailment and Normalisation
5. Computing Satisfaction

Formal Semantics of DL

- As in FOL, in *DL*, an *interpretation* ζ is a pair (δ, φ), where δ is any set of objects called the *domain* of the interpretation, and φ is a mapping called the *interpretation mapping* from the nonlogical symbols of *DL* to elements and relations over δ, *where*

    1. *for every constant c,* $\varphi(c) \in \delta$;
    2. *for every atomic concept a,* $\varphi(a) \subseteq \delta$;
    3. *for every role r,* $\varphi(r) \subseteq \delta \times \delta$

- Comparing DL to FOL, we can see that constants have the same meaning, atomic concepts are understood as unary predicates, roles as binary predicates.

Formal Semantics of DL

- As in FOL, in *DL*, an *interpretation* **ζ** is a pair (δ, φ), where δ is any set of objects called the *domain* of the interpretation, and φ is a mapping called the *interpretation mapping* from the nonlogical symbols of *DL* to elements and relations over δ, *where*

    1. *for every constant c,* $\varphi(c) \in \delta$;
    2. *for every atomic concept a,* $\varphi(a) \subseteq \delta$;
    3. *for every role r,* $\varphi(r) \subseteq \delta \times \delta$

- Comparing DL to FOL, we can see that constants have the same meaning, atomic concepts are understood as unary predicates, roles as binary predicates.

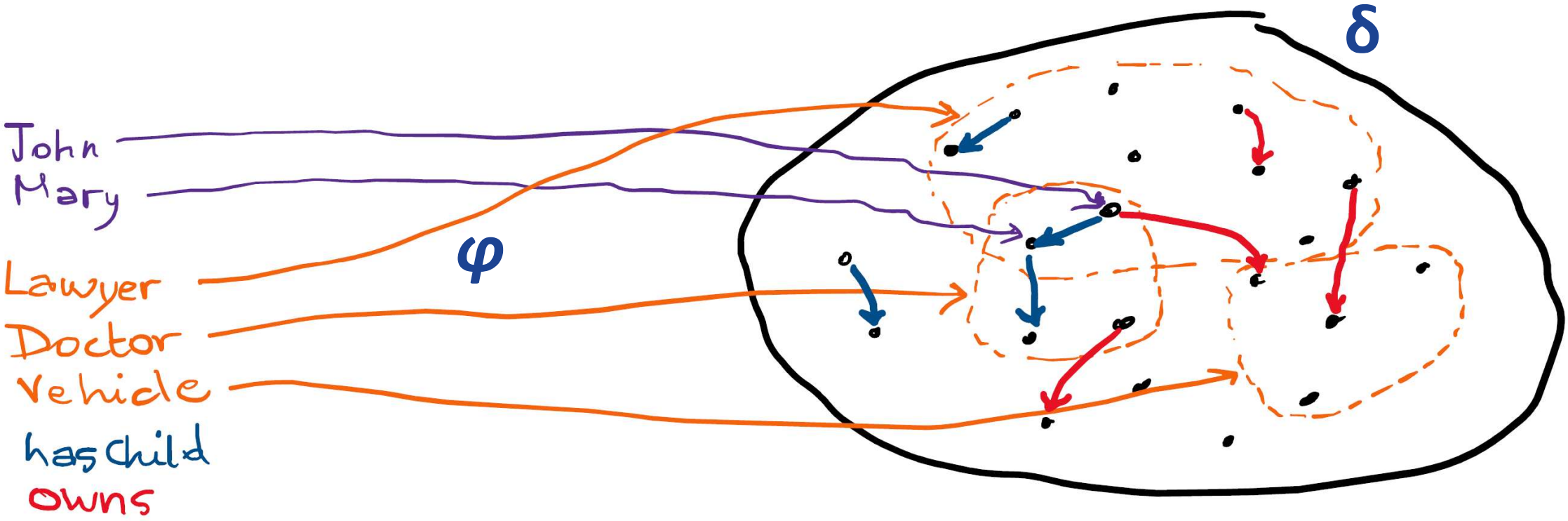
- A distinguishing feature of DL is the existence of nonatomic concepts whose meanings are completely determined by the meanings of their parts.

    - For example, the extension of [**AND** Doctor Female] is required to be the intersection of the extension of Doctor and that of Female.

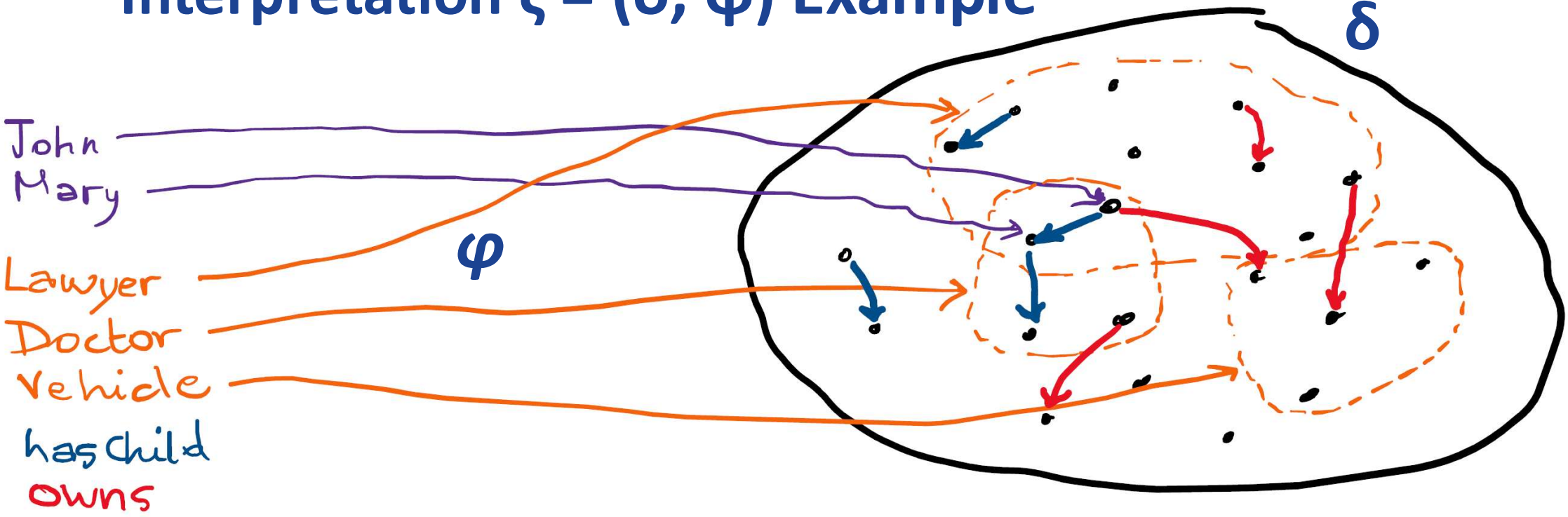
Formal Semantics of DL

- An *interpretation* $\boldsymbol{\zeta} = (\delta, \varphi)$ in DL satisfies:
  1. $a \sqsubseteq b$ iff $\varphi(a) \subseteq \varphi(b)$;
  2. $c(a)$ iff $\varphi(a) \in \varphi(c)$;
  3. $r(a, b)$ iff $(\varphi(a), \varphi(b)) \in \varphi(r)$

Interpretation ζ = (δ, ϕ) Example



Interpretation ζ = (δ, ϕ) Example



This interpretation satisfies: Lawyer(John), Lawyer(Mary), Doctor(John), Doctor(Mary), and hasChild(John,Mary), Doctor ⊑ Doctor, Lawyer ⊑ Lawyer, Vehicle ⊑ Vehicle

Formal Semantics of DL

- Extending the definition of **ζ** to all concepts.
- An *interpretation* **ζ** = (δ, φ) in DL satisfies:

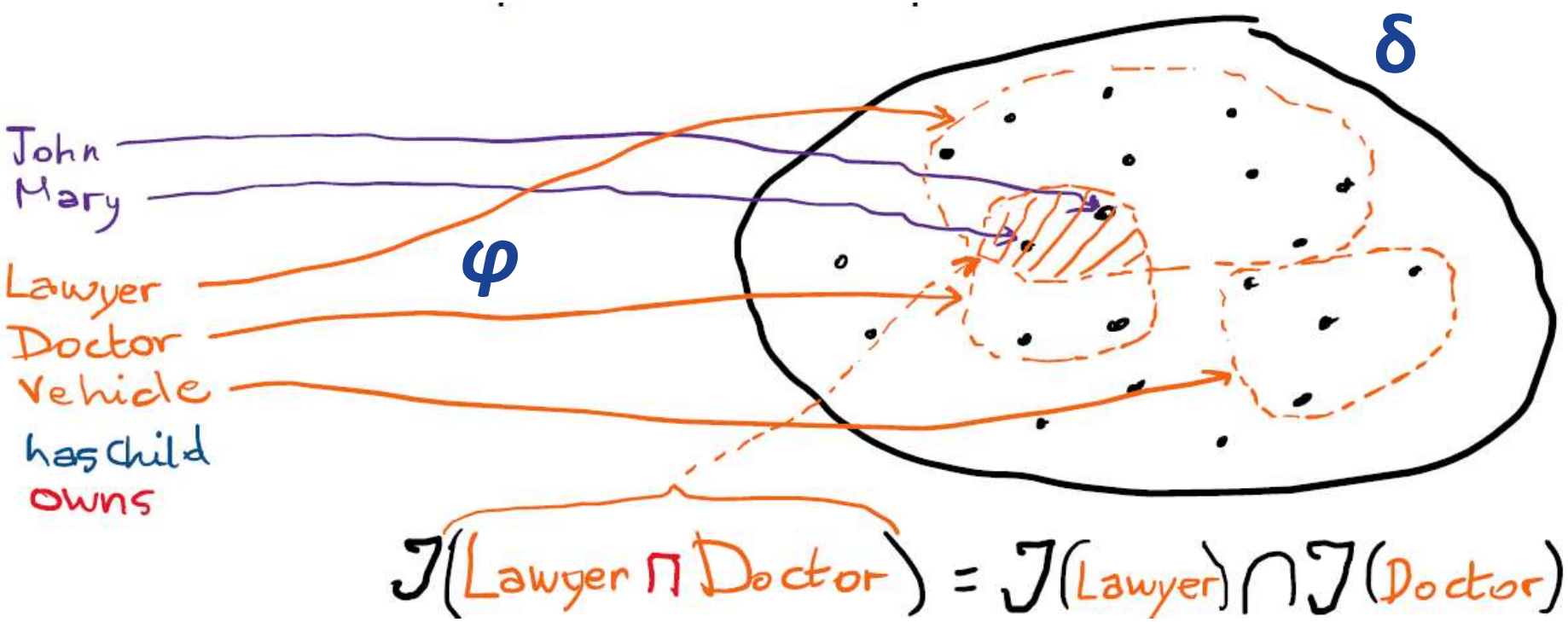
1. a $\sqsubseteq$ b iff φ(a) $\subseteq$ φ(b);
2. c(a) iff φ(a) $\in$ φ(c);
3. r(a, b) iff (φ(a), φ(b)) $\in$ φ(r);
4. φ([**ALL** r d]) iff {x $\in$ δ | for any y, if (x, y) $\in$ φ(r), then y $\in$ φ(d)};
5. φ([**EXISTS** n r]) iff {x $\in$ δ | there are at least n distinct y such that (x, y) $\in$ φ(r)}
6. φ([**FILLS** r c]) iff {x $\in$ δ | (x, φ(c)) $\in$ φ(r)};
7. φ([**AND** d_1 … d_n]) iff $\varphi(d_1) \cap \ldots \cap \varphi(d_n)$

Note that a $\doteq$ b iff a $\sqsubseteq$ b and b $\sqsubseteq$ a

DL Variation

- There are some variations in the way we write description logic formulas / sentences.
  - The concept of everything: $\top$ (Some literature refers this as **Thing**)
  - The empty concept: $\bot$
  - [ALL r d] can be written as $\forall r.d$
  - [EXISTS n r] is replaced with [EXISTS r d] which is written as $\exists r.d$
    - Note that $\exists r.d$ has closer meaning to $\forall r.d$ (with the difference on the quantifiers) than [EXISTS n r].
  - [AND d_1 … d_n] can be written as $d_1 \sqcap \ldots \sqcap d_n$
  - In some literature **OR** is also introduced, and it can be written as $d_1 \sqcup \ldots \sqcup d_n$
- Examples of these complex concepts used in sentences:
  - Father $\sqcap$ Mother $\sqsubseteq \bot$ (the concepts Father and Mother are disjoint)
  - Parent $\sqsubseteq \exists$hasChild.$\top$ (parents have – at least – a child)
  - $\top \sqsubseteq$ Good $\sqcup$ Bad (everything is either good or bad)

Interpretation ζ = (δ, φ) Example



Entailment in DL

- Given an interpretation $\zeta = (\delta, \varphi)$, a sentence $\boldsymbol{s} \in \boldsymbol{S}$, we say that $\boldsymbol{S}$ is true in ζ, written as $\zeta \models \boldsymbol{S}$ iff each sentence $\boldsymbol{s}$ is true in ζ (i.e. $\zeta \models \boldsymbol{s}$)
- Given a KB consisting of DL sentences, there are two basic sorts of reasoning we consider:
  - determining whether or not some constant c satisfies a certain concept d
    - KB $\models (c \rightarrow d)$
  - determining whether or not a concept d is subsumed by another concept d'
    - KB $\models (d \sqsubseteq d')$
- As in FOL, reasoning in a description logic means calculating entailments.

Entailment Examples

- In some cases the entailment relationship will hold because the sentences themselves are valid (i.e. $\models \boldsymbol{s}$). For example, consider the sentence:

  ([**AND** Doctor Female] $\sqsubseteq$ Doctor)

- This sentence is valid because it is always true in every interpretation ζ no matter what extension it assigns to Doctor and Female,
- The extension of the **AND** concept (which is the intersection of the two sets) will always be a subset of the extension of Doctor.
- Consequently, for any KB, the first concept is subsumed by the second—in other words, a female doctor is always a doctor.

Entailment Examples

- Similarly, the sentence

  (john → T)

- is valid. The sentence must be true in every interpretation because no matter what extension it assigns to john, it must be an element of δ, which is the extension of T (i.e. the concept of everything).
- Consequently, for any KB, the constant satisfies that concept—in other words, the individual John is always something.

Entailment Examples

- In more typical cases, the entailment relationship will depend on the sentences in the KB. For example, if a knowledge base, KB, contains the sentence

  (Surgeon $\sqsubseteq$ Doctor)

- Then we get the following entailment:

  KB |= ([**AND** Surgeon Female] $\sqsubseteq$ Doctor)

- Is this entailment true?

Entailment Examples

- To see why, consider any interpretation ζ, and suppose that ζ |= KB.
- Then, for this interpretation, the extension of Surgeon is a subset of that of Doctor, and so the extension of the **AND** concept (that is, the intersection of the extensions of Surgeon and Female) must also be a subset of that of Doctor.
- So for this KB, the **AND** concept is subsumed by Doctor: If a surgeon is a doctor (among other things), then a female surgeon is also a doctor.
- This conclusion would also follow if instead of (Surgeon ⊑ Doctor), the KB were to contain

  (Surgeon $\doteq$ [AND Doctor [FILLS :Specialty surgery]]).

- In this case we are defining a surgeon to be a certain kind of doctor, which again requires the extension of Surgeon to be a subset of that of Doctor.

Day 7
Description Logic

1. Structured Descriptions
2. A Description Language
3. Meaning and Entailment
4. **Computing Entailment and Normalisation**
5. Computing Satisfaction

Computing Entailment

- To compute Entailments, we will use the two basic sorts of reasoning we will be concerned with in description logics:
  - (c → d), where c is a constant and d is a concept
  - (d ⊑ e), where d and e are both concepts
- As with resolution for FOL, the key fact about the symbol-level computation that we are about to present is that it is correct (sound and complete) relative to the knowledge-level definition of entailment given earlier.
- Before computing entailments, a *normalisation* needs to be performed.
- It is similar in spirit to the derivation of normal forms like CNF in FOL.
- Here are the 6 steps to perform normalisation.

Normalisation

1. **Expand definitions**:
   - Any atomic concept that appears as the left-hand side of a sentence in the KB is replaced by its definition.
   - For example, if we have the following sentence in KB,

     (Surgeon $\doteq$ [**AND** Doctor [**FILLS** :Specialty surgery]]),
   - Then a concept [**AND** ... Surgeon ...] expands to

     [**AND** ... [**AND** Doctor [**FILLS** :Specialty surgery]] ...].

Normalisation

2. **Flatten the AND operators:**
   - Any subconcept of the form

     [**AND** ...[**AND** d_1 ... d_n]...]
   - can be simplified to

     [**AND** ... d_1 ... d_n ...]
3. **Combine the ALL operators:**
   - Any subconcept of the form

     [**AND** ...[**ALL** r d_1]...[**ALL** r d_2]...]
   - can be simplified to

     [**AND** ...[**ALL** r [**AND** d_1 d_2]] ...].

Normalisation

4. **Combine EXISTS operators:**
   - Any subconcept of the form

     [**AND** ...[**EXISTS** n_1 r]...[**EXISTS** n_2 r]...]
   - can be simplified to

     [**AND** ...[**EXISTS** n r] ...], where n is the max of n_1 and n_2.
5. **Deal with T:**

   Certain concepts are vacuous and should be removed as an argument to **AND**,

   e.g. [**AND** a b c T] should become [**AND** a b c].

   Some concepts should be removed altogether,

   e.g. [**ALL** r T] and [**AND**] (**AND** with no argument).
6. **Remove redundant expressions.**

Normalisation Example

- Consider the following example. Assume that KB has the following definitions:

  WellRoundedCo $\doteq$ [**AND**

  Company [**ALL** :Manager [**AND** B-SchoolGrad [**EXISTS** 1 :TechnicalDegree]]]

  ]

  HighTechCo $\doteq$ [**AND** Company [FILLS :Exchange nasdaq] [**ALL** :Manager Techie]]

  Techie $\doteq$ [**EXISTS** 2 :TechnicalDegree]

- These definitions amount to a WellRoundedCo being a company whose managers are business school graduates who each have at least one technical degree, a HighTechCo being a company listed on the NASDAQ whose managers are all Techies, and a Techie being someone with at least two technical degrees.

Normalisation Example

- Given these definitions, let us examine how we would normalize the concept

  [**AND** WellRoundedCo HighTechCo].

Normalisation Example

- Given these definitions, let us examine how we would normalize the concept

  [**AND** WellRoundedCo HighTechCo].

- First, we would expand the definitions of WellRoundedCo and HighTechCo, and then Techie, yielding:

```
[AND
  [AND Company
    [ALL :Manager [AND B-SchoolGrad
                       [EXISTS 1 :TechnicalDegree]]]]
  [AND Company
    [FILLS :Exchange nasdaq]
    [ALL :Manager [EXISTS 2 :TechnicalDegree]]]]
```

Normalisation Example

- Next, we flatten the AND operators at the top level and then combine the ALL operators over :Manager:

Normalisation Example

- Next, we flatten the AND operators at the top level and then combine the ALL operators over :Manager:

  [**AND** Company
  [**ALL** :Manager [**AND** B-SchoolGrad
  [**EXISTS** 1 :TechnicalDegree]
  [**EXISTS** 2 :TechnicalDegree]]]
  Company
  [**FILLS** :Exchange nasdaq]]

- What should we do next?

Normalisation Example

- Next, we flatten the AND operators at the top level and then combine the ALL operators over :Manager:

  [**AND** Company
  [**ALL** :Manager [**AND** B-SchoolGrad
  [**EXISTS** 1 :TechnicalDegree]
  [**EXISTS** 2 :TechnicalDegree]]]
  Company
  [**FILLS** :Exchange nasdaq]]

- Finally, we remove the redundant Company concept and combine the EXISTS operators over :TechnicalDegree, yielding the following:

  [**AND** Company
  [**ALL** :Manager [**AND** B-SchoolGrad [**EXISTS** 2 :TechnicalDegree]]]
  [**FILLS** :Exchange nasdaq]]

Day 7
Description Logic

1. Structured Descriptions
2. A Description Language
3. Meaning and Entailment
4. Computing Entailment and Normalisation
5. **Computing Satisfaction**

Computing Satisfaction

- In general, we wish to determine whether or not KB |= (b → e), where b is a constant and e is a concept. For example, if we have a KB that contains (b → d) and where KB |= (d ⊆ e), then we clearly have a case where KB |= (b → e).

- For example, suppose we have a KB that contains:

  joe → Person

  canCorp → [**AND** Company

  [**ALL** :Manager Canadian]

  [**FILLS** :Manager joe]]

- It is not hard to see that KB |= (joe → Canadian), even though the → sentence in the KB about joe does not lead us to this conclusion.

Summary

- There are also the concept of inheritance in Structured descriptions, which is very straightforward.
- In situations where the description logic language is expressive enough, a DL could in fact be used entirely to take the place of a production system.
- Recent work on OWL, which provides knowledge-structuring primitives that can be used to build ontologies for the Semantic Web, is based on description logics.

Until the next session:

- Download Protégé (the one that doesn't need installing) to your computer.